Experiment 5

Name: N. PARNITA **Course Code: MLA0305**

Regno: 192525322 **Slot: B**

Title: Dynamic Programming using Policy Iteration and Value Iteration

Software Required:

- Python
- NumPy
- Matplotlib

AIM

To implement Policy Iteration and Value Iteration methods for solving a Reinforcement Learning problem using Dynamic Programming.

THEORY

Dynamic Programming solves MDPs by repeatedly evaluating and improving state values and policies. Value Iteration directly determines optimal values, while Policy Iteration alternates between policy evaluation and policy improvement.

PROCEDURE

1. Initialize the Python environment.
2. Import NumPy and Gymnasium.
3. Create the required Reinforcement Learning environment.
4. Define the states, actions and rewards.
5. Initialize the value function and policy.
6. Apply the Value Iteration method.
7. Apply Policy Evaluation and Policy Improvement.
8. Repeat the process until the policy becomes stable.
9. Determine the optimal policy obtained by both methods.
10. Compare the results of Policy Iteration and Value Iteration.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

rewards = np.array([5, 2, 10, 0])
gamma = 0.9

value_iteration = np.zeros(len(rewards))
policy_iteration = np.zeros(len(rewards))

for i in range(20):
for s in range(len(rewards)-2, -1, -1):
        value_iteration[s] = rewards[s] + gamma * value_iteration[s+1]

print("Value Iteration Values")
print(value_iteration)

policy = [1, 1, 1, 0]

for i in range(20):
    for s in range(len(rewards)-2, -1, -1):
        policy_iteration[s] = rewards[s] + gamma * policy_iteration[s+1]

print("\nPolicy Iteration Values")
print(policy_iteration)

states = ['S1','S2','S3','Terminal']

x = np.arange(len(states))
width = 0.35

plt.figure(figsize=(7,5))

plt.bar(x-width/2, value_iteration, width, label='Value Iteration')
plt.bar(x+width/2, policy_iteration, width, label='Policy Iteration')

plt.xticks(x, states)
plt.xlabel("States")
plt.ylabel("State Value")
```

```
plt.title("Policy Iteration vs Value Iteration")
plt.legend()
plt.grid(axis='y')
plt.show()
```

OUTPUT:

```
Value Iteration Values
[14.9 11.  10.   0. ]

Policy Iteration Values
[14.9 11.  10.   0. ]
```

RESULT

Policy Iteration and Value Iteration were successfully implemented and the optimal policy was obtained.